Computing Consistent Levels for Layered Software Architecture Visualizations

Separating Class Depth, Package Order, and Local Layout

[Draft Version 3 – for co-author review]

Johannes Weigend
Weigend AM GmbH & Co.KG, Germany/Brazil
johannes.weigend@weigend.de

Michael Philippsen
Friedrich-Alexander-Universität Erlangen-Nürnberg, Germany
michael.philippsen@fau.de

Keywords: software architecture visualization, layout invariants, layered software city, strongly connected components, dependency analysis, software engineering tools

Abstract: Layered architecture visualizations communicate dependency structure through vertical position: elements that depend on others are placed higher. If level computation is wrong, the drawing can still look plausible while misleading the viewer. The central problem is not rendering, but defining the right ordering problem: class depth, package order, and local placement inside a package are related, but they are not the same hierarchy.

Our 2021 IVAPP paper, *A Layered Software City for Dependency Visualization* (Dashuber et al., 2021), introduced a city layout in which the X/Y position of class buildings follows dependency-derived levels, while building height along the Z axis encodes orthogonal metrics such as complexity or lines of code. S202 reimplements this dependency-level pipeline as an open-source Java tool under Apache 2.0 and uses the reimplementation to isolate what a correct level algorithm must compute.

The resulting solution computes three separate values on three separate graphs. First, a class architecture level is the longest path in the SCC-collapsed class dependency graph. Second, a package architecture level is computed on a weighted inter-package graph: package order follows the dominant direction of aggregated method-call traffic, not the deepest class contained in the package. Third, a local layer index is computed per parent package on the sibling-only dependency graph and determines the visual order of classes and subpackages inside that container. This separation avoids feedback loops in which class levels lift packages, package movement changes local order, and sorting no longer has a stable input graph. Highly cyclic code bases are handled by a weight-based SCC breaking heuristic that classifies back edges explicitly rather than hiding them.

We arrived at this separation through invariant checking. The Layout Invariant Checker was initially a debugging tool, but its findings exposed which postconditions a correct pipeline must satisfy: non-heuristic class dependencies respect class levels, dominant weighted package dependencies respect package levels, cyclic peer packages share a level after filtering, visual ranks follow local layer indices, and cached edge classifications match the current SCC and level state. The invariants therefore validate the algorithm; they are not a substitute for the three-level separation.

Two real systems demonstrate the effect. In the 92-module *software-ekg-7* system, an invariant finding exposed a missing package-SCC equalization step and triggered a pipeline fix. On Minecraft Forge 1.19.2, separating global architecture depth from local layout reduces the visual upward-edge count from roughly 6 000 to 3 600 by removing artifacts of a previous parent-filtered weight pipeline; the remaining upward edges represent real architecture violations. The contribution is therefore a corrected level-computation algorithm for layered architecture visualization, with invariants as machine-checkable evidence that the computed layout remains consistent.

Draft Version 3 | May 15, 2026

REFERENCES

Dashuber, V., Philippsen, M., and Weigend, J. (2021). A layered software city for dependency visualization. In *Proceedings of the 12th International Conference on Information Visualization Theory and Applications (IVAPP)*, VISIGRAPP 2021, pages 15–26. SCITEPRESS. Best Paper Award.

Eades, P., Lin, X., and Smyth, W. F. (1993). A fast and effective heuristic for the feedback arc set problem. *Information Processing Letters*, 47(6):319–323.

Karp, R. M. (1972). Reducibility among combinatorial problems. In Miller, R. E. and Thatcher, J. W., editors, *Complexity of Computer Computations*, pages 85–103. Plenum Press.

Appendix A – Why Naive Single-Pass Does Not Work: Failure Modes

The Core Problem: Two Coordinate Concepts, Four Constraints

A correct layered layout has to satisfy two coordinate concerns at once that are easy to conflate but answer different questions:

- *Analysis (global)* – where does this element sit in the dependency stack of the analysed system? This is the longest-path depth in the dependency graph and is best understood per element type: classes are levelled on the class graph, packages on the weighted inter-package graph. Both are global numbers.
- *Layout (local)* – where should the box for this element sit inside its parent container? This is decided by the dependency relationships among the parent's direct children only; refs that leave the parent are irrelevant to local position because the parent itself is positioned by the same logic one level up.

The two coordinate concepts are independent in principle and need to be kept independent in the implementation. Mixing them is the root cause of every failure mode below. Four invariants follow from this separation:

1. **Class-level analysis constraint (R1-algo):** If $A \to B$ denotes a class dependency, then $\mathrm{archLvl}(A) > \mathrm{archLvl}(B)$ after heuristic back-edges are removed. This checks the longest-path assignment in the class-dependency SCC-DAG.
2. **SCC constraint (R2):** Classes in a cyclic dependency group receive the same class architecture level; packages that form a cyclic group in the filtered package dependency graph share the same package architecture level.
3. **Package-order constraint (R3):** In the weighted inter-package dependency graph, when the call traffic from package P_A to P_B dominates the reverse, P_A ends up at a strictly higher package architecture level than P_B.
4. **Visual-rank constraint (R1-visual):** For every class dependency $A \to B$ that is not a heuristic back-edge, the rendered position of A in its parent box chain sits above B. The comparison is lexicographic on the tuple of local layer indices along the parent path, ending with the class's own local index. This is the statement represented by the drawing.

R1-algo and R3 check the algorithm; R1-visual and R2 check the picture. The failure modes that follow show why each obvious shortcut collapses one of these into the other and loses information.

Failure Mode 1: Cascading Level Updates

An algorithm iterates over classes and assigns levels. It first sees class A in package P_1, computes $\mathrm{level}(A) = 3$, and therefore sets $\mathrm{level}(P_1) = 3$. Later it reaches class B in package P_2 and discovers that A and B are in the same SCC and must share a level, now $\mathrm{level}(A) = \mathrm{level}(B) = 5$. This forces P_1 to be recomputed. If P_1 depends on P_3, P_3 may need to move as well. A cascade follows.

In a naive single-pass implementation, however, P_1 and P_3 may already have been finalized. SCC membership is only known after the relevant graph has been traversed; if level assignment and SCC discovery are interleaved, the iteration order decides whether the correction is still applied in time.

Failure Mode 2: Interleaved Cycle Detection Produces Classpath-Dependent Results

The most subtle practical failure mode occurs when cycle detection and level computation are not separated. The algorithm traverses the class graph, assigns levels, and handles discovered cycles on the fly, depending on which class happens to be processed first.

Classes come from JARs. The order in which the JVM sees classes follows the order in which they were packaged into the JAR. That order can depend on the build tool, module order in a multi-module build, filesystem ordering on the build server, and other implementation details. None of these orders is semantically meaningful; none encodes architectural dependency direction.

The result is unreliable: the same source code can produce different level layouts under different build configurations. The errors are subtle. The layout may still look plausible, arrows may mostly point in the expected direction, and no obvious visual break appears. Yet the precise hierarchy information is corrupted and may remain unnoticed for a long time.

The solution is to separate the phases strictly: first compute the SCC partition on the complete graph, then build the SCC-DAG, then assign levels. Tarjan's algorithm computes the SCC partition in $O(V+E)$; with deterministic input ordering, the implementation becomes reproducible and independent of classpath traversal artifacts.

Formal Counterexample: Why a Naive Recursive Algorithm Fails

The intuitive level-computation algorithm is a recursive DFS with memoization:

```
level(A):
  if A already computed: return level(A)
  if A has no dependencies: return 0
  return max(level(dep) for dep in A.dependencies) + 1
```

For acyclic graphs this is correct and equivalent to longest-path computation on a DAG. For cyclic graphs it fails structurally.

Counterexample with two SCCs:

```
SCC_1 = {A, B}, with A -> B -> A
SCC_2 = {C, D}, with C -> D -> C
Cross dependency: A -> C
```

The correct levels are $\text{level}(C) = \text{level}(D) = 0$ and $\text{level}(A) = \text{level}(B) = 1$.

Recursive algorithm, starting at A:

```
level(A)  [A "in progress", sentinel = 0]
  -> level(B)
      -> level(A): cycle detected, return 0
    B.level = 0 + 1 = 1
  -> level(C)  [C "in progress", sentinel = 0]
      -> level(D)
          -> level(C): cycle detected, return 0
        D.level = 0 + 1 = 1
    C.level = 1 + 1 = 2   <-- wrong; correct would be 0
  A.level = max(1, 2) + 1 = 3   <-- wrong; correct would be 1
```

After SCC equalization, $\text{SCC}_1 = \max(3,1) = 3$ and $\text{SCC}_2 = \max(2,1) = 2$. Both results are wrong.

The cause is that the recursive algorithm enters SCC_2 from the context of SCC_1. The sentinel-based local cycle handling inflates the level of SCC_2, and that inflated level then feeds into the computation of SCC_1. The correct level of SCC_2, namely zero, is only known after the graph has been collapsed into SCCs and evaluated as an SCC-DAG.

Thus, recursive level algorithms that mix traversal, cycle handling, and level assignment can produce traversal-order-dependent results on graphs with interacting SCCs. In JVM-based tools, that traversal order can be influenced by classpath and packaging order. The correct solution collapses SCCs before level assignment: Tarjan, then SCC-DAG, then longest path.

Failure Mode 3: Deriving Package Levels from Class Levels

An obvious implementation assigns each package a level equal to the maximum level of its contained classes, then lifts parent packages transitively. This is wrong for two reasons.

First, the rule conflates containment with dependency. A package that *contains* a class at level 7 is not necessarily architecturally positioned at level 7 itself; it may have no cross-package dependencies at all. Two disconnected package trees processed from the same JAR would be ordered relative to each other by accident of their class contents, not by their actual dependency relationships.

Second, the approach creates a circular dependency between the two level systems: class levels depend on the class graph, but package levels derived from class levels then constrain the visual position of packages relative to classes, making the combined coordinate system ill-defined.

S202 avoids this by computing class levels and package levels on independent graphs with no mutual constraint. Each system starts at zero and grows according to its own dependency structure.

Failure Mode 4: Large SCCs Without Heuristics Collapse the Layout

Without a mechanism for breaking large SCCs, all members of a cyclic group must be placed on the same level. For small cycles of two to five classes this is acceptable. For Minecraft Forge 1.19.2 (5 123 classes), however, we observed a single SCC containing 2 857 classes. Without a breaker, more than 55 percent of the project would be assigned the same level. All remaining classes would be distributed above, but the internal dependency hierarchy within this large cycle would be invisible.

The heuristic SCC breaker solves this practical problem. It identifies back edges—dependencies that run against the inferred architecture direction—and removes them for level computation. The removed edges are visualized as violations instead of being ignored. The layout preserves visibility of cyclic dependencies while still producing a usable hierarchy.

Relation to Feedback Arc Set Heuristics

Failure Mode 4 is an instance of the Minimum Feedback Arc Set problem: remove a minimal set of directed edges so the graph becomes acyclic. The problem is NP-hard (Karp, 1972), so any practical polynomial-time implementation must use a heuristic or approximation.

A well-known related heuristic is due to Eades, Lin, and Smyth (Eades et al., 1993). It orders nodes according to the difference between outgoing and incoming degree, then classifies edges that point backward in the resulting sequence as feedback edges. The intuition is similar to S202: high out-degree suggests a higher-level element; high in-degree suggests a lower-level element.

S202 uses a deliberately simpler variant: a direct normalized score per node,

$$r(P) = \frac{\text{outWeight}(P) - \text{inWeight}(P)}{\max(1,\ \text{outWeight}(P) + \text{inWeight}(P))},$$

where the weights are summed within the SCC under consideration. At the class level the weights are the in- and out-degrees in the class dependency graph; at the package level they are aggregated method-call counts. An edge $P_A \to P_B$ inside an SCC is classified as a back-edge candidate when $r(P_A) < r(P_B) - 0.1$. Equal-rank pairs represent symmetric bidirectional dependencies with no dominant architectural direction; they remain in the same SCC and are assigned the same level.

The important distinction is transparency. Many tools break cycles implicitly, but S202 classifies heuristic cut edges explicitly. They are marked as `VIOLATION`, rendered in red, and known to the invariant checker. This makes algorithmic defects separable from tolerated heuristic artifacts.

The Solution: Three Coordinate Concepts on Independent Graphs

Failure modes 1 to 3 share a root cause: conflating coordinate concepts that belong to different semantic domains. S202 resolves this by computing three independent coordinate values, two of them for analysis and one for layout.

- **Class architecture level** is the longest path in the SCC-collapsed class dependency DAG. It represents global architectural depth: how many layers of dependencies separate a class from the bottom of the analysed dependency graph.
- **Package architecture level** is the longest path in the SCC-collapsed weighted inter-package dependency graph. Edge weights are method-call counts aggregated from class-level references. The result represents the module's position in the system's overall dependency order, independently of the class-level coordinate.
- **Local layer index** is the longest path in the SCC-collapsed sibling-only dependency graph of a single parent container. The graph contains only edges whose source and target both lie in some direct child of that parent; references leaving the parent's subtree are excluded because they are handled by the enclosing container.

Every element (class and package) gets a local layer index that is only meaningful within its own parent box. The renderer sorts each parent's children by this index, so the visible Y position of a box is the local layer index, not the global architecture level.

All three concepts use the same algorithmic shape—Tarjan SCC detection, weight-based back-edge identification, DAG condensation, longest-path propagation—but on three different graphs with three different scopes. Because the systems are independent, packages with no cross-package dependencies correctly land at architecture level 0 regardless of the class levels inside them, and a top-level class like *ArchitectureView* that orchestrates a sub-package such as *ui.rendering* sits visually above the sub-package box without distorting either of their architecture levels.

The separation is an engineering choice driven by semantic clarity. A unified single-coordinate approach is theoretically possible, but it conflates three semantically distinct concerns—global architectural depth, module dependency order, and per-container visual position—into one number, making the pipeline hard to reason about, test, and maintain. Each simplification of such a unified approach turns into one of the failure modes above.

The three concepts have the following algorithmic structure. Phases 1 and 2 compute the global architecture levels; Phase 3 derives the local layer index per parent container from the architecture-level output.

Algorithm 1: Phase 1 – Global Class Levels (`LevelCalculator`, Step 3)

class dependency graph
→ Tarjan (SCCs)
→ SCCBreaker: identify heuristic back-edges in large SCCs ($|S| \geq 3$; 2-class SCCs are equalized instead)
$\text{rank}(P) = (\text{outDeg} - \text{inDeg}) \,/\, \max(1, \text{outDeg} + \text{inDeg})$
edge $A \to B$ is a back-edge candidate if $\text{rank}(A) < \text{rank}(B) - 0.1$
→ Remove back-edges → filtered class DAG
→ Condense into SCC-DAG → longest-path pass
→ Assign SCC level to all member classes (equalization implicit)

Algorithm 2: Phase 2 – Package Architecture Levels (`LevelCalculator`, Step 4)

Build weighted inter-package dependency graph $G_P = (V_P, E_P, w)$:
For each class C in package P_{leaf} with method calls to class D in P_{target} ($P_{\text{leaf}} \neq P_{\text{target}}$):
$w(P_{\text{ancestor}}, P_{\text{target}}) \ += \ \text{callCount}(C \to D)$
for every ancestor P_{ancestor} of P_{leaf} with $P_{\text{ancestor}} \neq P_{\text{target}}$
Fallback weight $= 1$ for structural references with no method-call data
No directional filtering: both parent→child and child→ancestor edges enter G_P. The SCC breaker decides direction by weight.

Iterative SCC-breaking (repeat until stable):
Tarjan on current graph → find SCCs with $|S| \geq 2$ (2-package SCCs broken if rank differs; equalized otherwise)

For each member $P \in S$:
$$r(P) = \left(\sum_{Q \in S} w(P,Q) - \sum_{Q \in S} w(Q,P)\right) \ / \ \max\left(1, \sum_{Q \in S} w(P,Q) + \sum_{Q \in S} w(Q,P)\right)$$
Edge $P_A \to P_B$ inside S is a back-edge if $r(P_A) < r(P_B) - 0.1$; remove it
Equal-rank pairs remain in the SCC → assigned the same level

Level assignment:
Condense remaining graph into SCC-DAG
Longest-path propagation on SCC-DAG
Assign SCC level to all member packages

Phase 1 produces classpath-independent class architecture levels because SCCs are collapsed before levels are assigned. Phase 2 produces package architecture levels directly from package dependencies: every class reference contributes to the weighted graph regardless of the containment relation between source and target package, and a per-SCC rank score decides which direction of a cyclic group dominates. The two architecture coordinate systems do not constrain each other—a package with no outgoing cross-package calls lands at level 0 regardless of the class levels inside it, and conversely the class levels of a package's contents do not lift the package itself.

Algorithm 3: Phase 3 – Local Layer Index per Parent Container (`LocalLayerCalculator`, Step 6)

for each parent package P with at least two direct children:
  Let $S(P)$ be the direct children (classes and sub-packages).
  Build sibling-only weighted graph G_P^{loc} on $S(P)$:
    edge $X \to Y$ with weight $\sum \text{callCount}(c \to d)$
    over all class refs $c \to d$ where c lies in the subtree of sibling X
    and d lies in the subtree of sibling Y;
    refs leaving the parent's subtree are excluded.
  Tarjan on $G_P^{\text{loc}} \to$ SCCs.
  Within each SCC of size ≥ 2: weight-based rank break (same formula as Phase 2).
  Condense to SCC-DAG, longest-path pass, assign **localLayerIndex** per sibling.

Phase 3 is a pure layout phase. It never modifies architecture levels. Each parent container is computed independently: a class's local layer index inside *ui* carries no relationship to a class's local layer index inside *ui.rendering*. The renderer sorts each parent's children by their local layer index, descending. Cross-container ordering is handled by the enclosing parent's local layer index, applied recursively. Local cycles—two sibling sub-packages with mutual class references—are resolved silently by the rank break; they are not surfaced as a separate Tangle category because the user-visible tangle list remains a global architecture statement.

	Phase 1 – Class	Phase 2 – Package	Phase 3 – Local Layer
Graph	Class dependency	Weighted inter-package	Sibling-only weighted (per parent)
Edge weight	Unweighted	Method-call counts	Method-call counts
Scope	Global	Global	Per parent container
Meaning	Class arch. depth	Module dep. position	Visual Y in parent box
Zero point	Class with no deps	Package with no outgoing calls	Bottom sibling in box
Cycles	SCCBreaker (degree)	Weight-rank, all edges	Weight-rank within siblings
Used by	R1-algo, Tangles, R2 (class)	R3, R2 (package)	R1-visual, renderer sort

Table 1: The three coordinate concepts computed by S202.

Layout Invariants for Level Correctness

The three level phases define five machine-checkable postconditions verified after every analysis run. R1-algo, R2, R3, R5 are checked by the `LayoutInvariantChecker` as algorithm consistency checks; R1-visual is checked by the architecture-builder's violation detection on the visual-rank tuple.

Rule	Postcondition
R1-algo	Non-heuristic class dependencies respect class architecture levels: $A \to B \Rightarrow \text{archLvl}(A) > \text{archLvl}(B)$. Tests the longest-path assignment of Phase 1.
R1-visual	For every non-heuristic class dependency $A \to B$, the rendered position of A sits above B. Position is compared lexicographically on $[L(\text{ancestor}_1), \ldots, L(\text{ancestor}_n), L(A)]$ versus $[L(\text{ancestor}'_1), \ldots, L(\text{ancestor}'_m), L(B)]$ where L denotes the local layer index. Covers same-parent, same-grandparent, and cross-tree comparisons uniformly.
R2	Packages forming a cyclic group in the filtered package dependency graph share the same package architecture level.
R3	For every weighted package edge (P_A, P_B) with $w(P_A, P_B) > w(P_B, P_A)$ that is not a back-edge cut by the SCC breaker: $\text{archLvl}(P_A) > \text{archLvl}(P_B)$.
R5	Cached edge classifications (`NORMAL`, `VIOLATION`, `INTRA_SCC`) remain consistent with the current level assignment and SCC membership.

Table 2: Five layout invariants verified after every analysis run. R1-algo, R3 and R2 check the algorithm; R1-visual checks the rendered picture; R5 keeps the cached annotations consistent.

Throughout the iterative development of the level pipeline, R2 surfaced implementation bugs after algorithm changes that the unit-test suite had not detected. Figure 1 shows a representative R2 report. In *software-ekg-7*, for instance, an R2 finding exposed a missing package-SCC equalization step and directly triggered a pipeline fix.

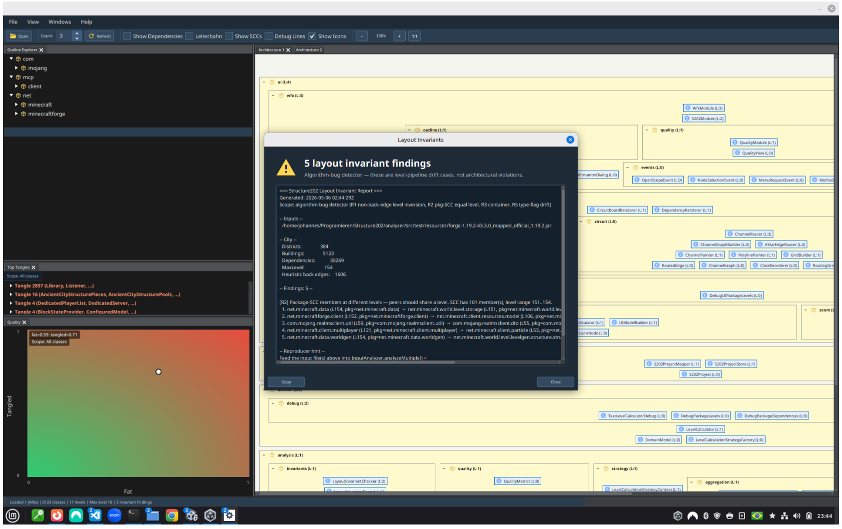


Figure 1: R2 checker report: packages in a filtered package-SCC are placed on different levels. Findings are reported as algorithm-bug candidates with a reproducer block, which makes the visual failure traceable to concrete level-pipeline constraints.

R2 also guided refinement of the rule itself. Against Minecraft Forge (2 895 classes, 220 packages, 188 of 207 packages in a single package-level SCC), two R2 reports turned out to be *false positives*: R2 filtered class-level heuristic back-edges but not the package-level back-edges that the package SCC-breaker had already removed. R3 already applied the correct filter; once R2 was aligned, the Minecraft findings disappeared and all 155 invariant tests pass. The invariant checker connects the visual symptom to a reproducible algorithm defect.

The Remaining Rules: R1-algo, R1-visual, R3, and R5

R1-algo ($A \to B \Rightarrow \mathrm{archLvl}(A) > \mathrm{archLvl}(B)$ on the filtered class graph) is the most direct invariant on the analysis side: it is a corollary of longest-path computation on the SCC-DAG and holds trivially for any correctly implemented class-level assignment. Its value lies in providing an immediate check—a failing R1-algo signals a fundamental breakdown in the class-level pipeline.

R1-visual (lexicographic comparison of local-layer-index tuples along the parent chain) is the invariant represented by the drawing. The two checks are intentionally separated: R1-algo asks "did Phase 1 compute consistent class architecture levels?", R1-visual asks "does the rendered Y position reflect the class dependencies?" Both pass on a correct implementation; only the second one is allowed to fail when the sibling-only graph of some parent exposes a genuine upward edge, because that is a real architecture violation rather than an algorithm bug. The single visual-rank comparison covers every pair: siblings inside the same container compare on their final tuple element; cross-tree pairs diverge earlier; classes versus sub-package boxes in the same parent compare on the shared parent context. No special-case logic is needed for parent→child versus sibling versus cross-package class references.

R3 verifies that the dominant direction in the weighted package dependency graph is reflected in the architecture level assignment. For any package edge (P_A, P_B) where $w(P_A, P_B) > w(P_B, P_A)$—meaning P_A calls into P_B significantly more than vice versa—the invariant requires $\mathrm{archLvl}(P_A) > \mathrm{archLvl}(P_B)$. Package back-edges identified during SCC-breaking are excluded from this check, analogously to how R1-algo excludes class-level heuristic back-edges. A failing R3 indicates that the weight-based SCC-breaking cut the wrong package edge, or that the package-level DAG propagation did not converge correctly.

R5 validates that every edge's stored classification (`NORMAL`, `VIOLATION`, `INTRA_SCC`) is still consistent with the current level assignment and SCC membership. Edge labels are computed during analysis and cached in the model; after any algorithmic change they can silently become stale. R5 is therefore a cross-cutting consistency check: it ties the structural output of the level pipeline back to the semantic annotations visible to the user.

R4: A Retired Rule and What It Teaches

An early version of the invariant checker contained R4:

```
Cyclic child packages must share a level.
```

The rule sounds reasonable and is correct in simple cases. In practice it failed because the raw package graph contains relationships that should not be interpreted as peer-level architectural cycles:

- **Parent-child edges** create apparent bidirectional relationships because a parent contains its children and children belong to their parent. Without filtering, many parent/child pairs look cyclic.
- **SCCBreaker back edges** connect packages that are not necessarily architectural peers. A rule that includes these edges equalizes the wrong candidates.
- **Shared class SCCs** can make two packages appear package-dependent without representing a true structural peer relationship.

R4 was replaced by R2, which expresses the same intention on the correctly filtered graph. R2 removes class-level heuristic back-edges, package-level SCC-breaking back-edges, intra-subtree edges, and edges caused by shared class SCCs before running Tarjan on the remaining package graph.

Specifying invariants for a three-level constraint system turns out to be harder than it appears. The checker therefore provides a second safety net: it not only tests the implementation against the rules, but also exposes when the rules themselves are too broad or too coarse. False positives force the rule to become more precise. The later R2 finding in *software-ekg-7* shows that the refined rule was precise enough to distinguish real pipeline bugs from apparent ones.

Implementation: The Complete Pipeline

The conceptual structure expands into the following concrete implementation steps, which constitute the reference implementation (Appendix B).

Algorithm 1: Complete Level Pipeline (`LevelCalculator`)

Step 1: Create class objects (all levels $\leftarrow 0$)
Step 2: Create package objects (all levels $\leftarrow 0$)

Step 3: Compute class architecture levels (Phase 1)
$\rightarrow$ Tarjan on class dependency graph
$\rightarrow$ SCCBreaker: identify heuristic back-edges in large SCCs ($|S| \geq 3$):
$r(C) = (\text{outDeg} - \text{inDeg}) \,/\, \max(1, \text{outDeg} + \text{inDeg})$;
edge $A \rightarrow B$ is a back-edge candidate if $r(A) < r(B) - 0.1$
$\rightarrow$ Remove identified back-edges $\rightarrow$ filtered class DAG
$\rightarrow$ Condense into SCC super-nodes
$\rightarrow$ Single longest-path pass on condensed DAG
$\rightarrow$ Assign SCC level to all member classes

Step 4: Compute package architecture levels (Phase 2)
4a. Build weighted inter-package dependency graph:
$w(P_A, P_B) = \sum$ method-call counts from subtree(P_A) to P_B;
propagate weights up to all ancestor packages (no directional filtering);
both parent$\rightarrow$child and child$\rightarrow$ancestor edges enter the graph

4b. Iterative package SCC-breaking (repeat until stable):
Tarjan on current graph $\rightarrow$ SCCs with $|S| \geq 2$;
$r(P) = (\sum_{Q \in S} w(P,Q) - \sum_{Q \in S} w(Q,P)) \,/\, \max(1, \ldots)$;
edge $P_A \rightarrow P_B$ is a back-edge if $r(P_A) < r(P_B) - 0.1$; remove it;
equal-rank pairs remain in the same SCC $\rightarrow$ assigned the same level

4c. Level assignment on condensed package SCC-DAG:
longest-path propagation; assign SCC level to all member packages.
No child–parent lift: layout positioning is based on `localLayerIndex`.

Step 5: Set reverse dependencies (dependents)

Step 6: Compute local layer index per parent (Phase 3)
for each parent package P with two or more direct children:
build sibling-only weighted graph G_P^{loc} from class refs
whose source and target both live in some direct child of P;
Tarjan $\rightarrow$ rank-based SCC break $\rightarrow$ longest-path;
assign `localLayerIndex` to each direct child of P.

After Step 3, a class's architecture level is the level of the SCC node to which it belongs, measured as the longest path in the cycle-free quotient graph across all classes of the analysed project. Cycles do not influence level assignment internally because they have been collapsed before longest-path computation begins.

Step 4 treats class architecture levels and package architecture levels as independent coordinate systems. A package with no outgoing cross-package calls receives architecture level 0, regardless of the class levels inside it; two package trees with no shared dependencies are each levelled from zero in their own connected component of the weighted package graph. The earlier child–parent lift, which inflated parent package levels by the class levels their children called, is gone – it was a workaround for the missing layout coordinate.

Step 6 introduces the layout coordinate. Each parent container is treated as a self-contained scope: only class references that stay within the parent's subtree contribute to the local sibling graph. The result is that the box for *ArchitectureView* in the *ui* package sorts above the box for the *ui.rendering* sub-package, simply because *ArchitectureView* has heavier outgoing call traffic into rendering than rendering has back into *ui*. None of this disturbs Phase 1 or Phase 2.

Algorithm 2: Phase 4 – Horizontal Row Ordering (`HorizontalRowLayoutOptimizer`)

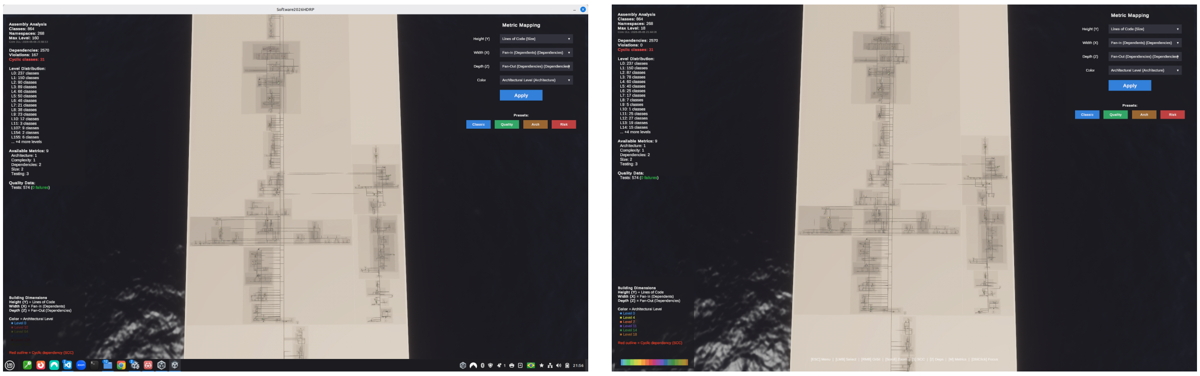
for each package P:
- **Step 1:** Group direct children by their `localLayerIndex` value
- **Step 2:** Collect subtree classes per row element
- **Step 3:** Build weighted undirected proximity links from inter-sibling class dependencies
- **Step 4:** Run bounded barycentric sweeps across rows
- **Step 5:** Store the result in `horizontalLayoutOrder`

rendering: sort a copied child view by `localLayerIndex` (Phase 3), then `horizontalLayoutOrder` (Phase 4), then full name

Phase 4 does not change any level or layer assignment and is deliberately excluded from the layout invariants. Its only purpose is cosmetic: when an overlay of dependency arrows is displayed, same-row elements that share many inter-sibling calls are placed horizontally near each other so the arrows are shorter and less crossed. The implementation stores this as a separate `horizontalLayoutOrder` field and never mutates the semantic child list.

R2 Fix in *software-ekg-7*: Visual Evidence

The *software-ekg-7* case is the concrete failure that motivated the package-level pipeline. Before the fix, the city view still looked plausible: the package rectangles and dependency edges formed a readable layout, and no single visual element made the pipeline defect obvious. The metrics panel, however, exposed the inconsistency: 167 dependency violations were reported and the maximum architectural level had inflated to 160. After the package-level equalization was corrected, the same input produced zero violations and a maximum level of 18. The number of classes, packages, dependencies, cyclic classes, and tests stayed constant; the change is therefore attributable to level propagation, not to a different input model.



(a) Before the fix: the full city view looks plausible, but the metric panel reports 167 violations and MaxLevel 160.

(b) After the fix: the same analyzed system reports zero violations and MaxLevel 18.

Figure 2: Full-screen before/after screenshots of the *software-ekg-7* software city. The visible city changes only subtly; the level metrics reveal the correction.

(a) Before: 167 violations, MaxLevel 160.

(b) After: zero violations, MaxLevel 18.

Figure 3: Cropped metric panels from Figure 2. These panels are the clearest user-visible symptom of the propagation defect and its fix.

Summary

Correct level computation for layered architecture visualizations requires keeping two coordinate concepts separate: *analysis* (where does this element sit in the global dependency stack?) and *layout* (where should its box go inside its parent container?). Every straightforward single-pass approach over a unified graph collapses these two questions into one coordinate, and the failure modes documented above show why each obvious simplification breaks down in practice.

S202 resolves this by computing three coordinate values on three independent graphs. *Class architecture levels* are the longest path in the SCC-collapsed class dependency DAG; they represent global architectural depth. *Package architecture levels* are derived from a weighted inter-package dependency graph whose edges carry aggregated method-call counts; they represent module dependency position independently of class levels. *Local layer indices* are computed per parent container on the sibling-only graph closed within that parent's subtree; they decide visual Y position inside each box without disturbing either architecture coordinate. All three use the same algorithmic shape: Tarjan SCC detection, weight-based back-edge identification, DAG condensation, longest-path propagation.

A horizontal row-ordering pass (Phase 4) improves the legibility of dependency overlays without modifying any coordinate or affecting any layout invariant. The Layout Invariant Checker verifies five machine-checkable postconditions (R1-algo, R1-visual, R2, R3, R5) after every analysis run and emits reproducer blocks for each finding.

Two cases demonstrate the value of the separation. In *software-ekg-7*, a plausible city layout reported 167 dependency violations and a maximum architectural level of 160 before a package-level equalization fix; after the fix, the same input produced zero violations and a maximum level of 18 with no change to the analysed source code. On Minecraft Forge 1.19.2, introducing the local layer index as a distinct layout coordinate and lifting the parent filter from the weighted package graph reduced the visual upward-edge count from roughly 6 000 to 3 600; the eliminated edges were artifacts of forcing parent packages below their sub-packages to keep them out of the algorithm's tangle list, and the remaining edges expose real architecture violations that deserves a code-level fix.

Appendix B – LevelCalculator Reference Implementation

The following source excerpt is the complete Java implementation of the level pipeline discussed above.

```java
package de.weigend.s202.domain.architecture;

import de.weigend.s202.domain.DomainModel;
import de.weigend.s202.domain.strategy.LevelCalculationStrategyContext;
import de.weigend.s202.graph.StronglyConnectedComponent;
import de.weigend.s202.graph.TarjanSCCFinder;
import de.weigend.s202.reader.DependencyModel;

import java.util.*;
import java.util.logging.Logger;

/**
 * Calculates architectural levels for classes and packages based on dependencies.
 *
 * Pipeline:
 *   Step 1  Create class objects       (all levels = 0)
 *   Step 2  Create package objects     (all levels = 0)
 *   Step 3  Compute class levels       strategy (Tarjan → SCC-break → DAG → longest-path)
 *   Step 4  Compute package levels     weighted inter-package graph → SCC-break → DAG → longest-path
 *   Step 5  Set reverse dependencies
 *   Step 6  Assign local layer index   per-parent sibling graph → SCC-break → DAG → longest-path
 *                                      (rendering position within each parent box, no global meaning)
 *
 * Package levels are computed independently from class levels using a weighted
 * inter-package dependency graph. The weight of an edge P_A → P_B is the
 * aggregated method-call count from classes in P_A's subtree to classes in P_B,
 * with a fallback weight of 1 for structural references without method-call data.
 * This separates the containment hierarchy from the dependency hierarchy and
 * correctly handles disconnected package trees (they are levelled independently,
 * both starting at 0).
 *
 * Package SCC-breaking uses a weight-based rank score:
 *   rank(P) = (Σ outgoing weights within SCC - Σ incoming weights within SCC)
 *             / max(1, sum of both)
 * Edge P_A→P_B is a back-edge when rank(P_A) < rank(P_B) - 0.1.
 * Equal-rank pairs (symmetric dependency) are left in the same SCC and assigned
 * the same level -- they are genuine peers with no dominant dependency direction.
 */
public class LevelCalculator {

    private static final Logger LOG = Logger.getLogger(LevelCalculator.class.getName());
    private static final double RANK_THRESHOLD = 0.1;

    private final LevelCalculationStrategyContext strategyContext;

    public LevelCalculator() {
        this(LevelCalculationStrategyFactory.createDefault());
    }

    public LevelCalculator(LevelCalculationStrategyContext strategyContext) {
        Objects.requireNonNull(strategyContext, "strategyContext cannot be null");
        this.strategyContext = strategyContext;
    }

    public DomainModel calculate(DependencyModel rawModel) {
        DomainModel model = new DomainModel();

        // Step 1: Create class objects (all levels = 0)
        for (String className : rawModel.getAllClassNames()) {
```

```java
            DependencyModel.ClassInfo rawClass = rawModel.getClass(className);
            model.addClass(className, new DomainModel.CalculatedElementInfo(
                className, rawClass.simpleName, "CLASS", 0,
                new HashSet<>(rawClass.dependencies), rawClass.interfaceType));
        }

        // Step 2: Create package objects (all levels = 0)
        for (String packageName : rawModel.getAllPackageNames()) {
            DependencyModel.PackageInfo rawPkg = rawModel.getPackage(packageName);
            model.addPackage(packageName, new DomainModel.CalculatedElementInfo(
                packageName, rawPkg.simpleName, "PACKAGE", 0, new HashSet<>()));
        }

        // Step 3: Compute class levels
        calculateClassLevels(model);

        // Step 4: Compute package levels from weighted inter-package graph.
        calculatePackageLevels(model, rawModel);

        // Step 5: Set reverse dependencies
        updateDependentRelationships(model);

        // Step 6: Assign per-parent local layer index -- independent of the
        // global architectureLevel, used by the renderer to position
        // siblings within each parent's box.
        new LocalLayerCalculator().assign(model, rawModel);

        return model;
    }

    public LevelCalculationStrategyContext getStrategyContext() {
        return strategyContext;
    }

    // ------------------------------------------------------------------------
    // Step 3 -- class levels
    // ------------------------------------------------------------------------

    private void calculateClassLevels(DomainModel model) {
        Map<String, Set<String>> classDeps = new HashMap<>();
        for (DomainModel.CalculatedElementInfo c : model.getAllClasses().values()) {
            classDeps.put(c.fullName, new HashSet<>(c.dependencies));
        }
        Map<String, Integer> levels =
            strategyContext.getClassLevelStrategy().calculateClassLevels(classDeps);
        for (Map.Entry<String, Integer> e : levels.entrySet()) {
            DomainModel.CalculatedElementInfo info = model.getClass(e.getKey());
            if (info != null) info.setArchitectureLevel(e.getValue());
        }
    }

    // ------------------------------------------------------------------------
    // Step 4 -- package levels via weighted inter-package graph
    // ------------------------------------------------------------------------

    private void calculatePackageLevels(DomainModel model, DependencyModel rawModel) {
        if (model.getAllPackages().isEmpty()) return;
        Set<String> allPkgNames = model.getAllPackages().keySet();

        // Build weighted graph: weight[from][to] = aggregated method-call
        // count from classes in 'from' to classes in 'to', with fallback
        // weight 1 for structural dependencies without method-call data.
```

```
        Map<String, Map<String, Integer>> weights = buildWeightedPackageGraph(model, rawModel);

        // Unweighted adjacency for Tarjan (direction matters, zero-weight edges excluded)
        Map<String, Set<String>> graph = new LinkedHashMap<>();
        for (String pkg : allPkgNames) graph.put(pkg, new LinkedHashSet<>());
        for (Map.Entry<String, Map<String, Integer>> entry : weights.entrySet()) {
            graph.get(entry.getKey()).addAll(entry.getValue().keySet());
        }

        // Iteratively break SCCs using weight-based rank, then assign levels
        // via SCC-collapsed DAG longest-path. Remaining SCCs (equal-rank peers)
        // are collapsed to the same level without cutting any edge.
        Set<String> backEdgeKeys = new LinkedHashSet<>(); // "from\0to" -- tracked for R3
        boolean changed = true;
        while (changed) {
            changed = false;
            for (StronglyConnectedComponent scc : new TarjanSCCFinder(graph).findSCCs()) {
                if (scc.getSize() < 2) continue;
                Set<String> members = scc.getMembers();

                // Compute weight-based rank for each member within this SCC
                Map<String, Double> rank = new HashMap<>();
                for (String m : members) {
                    int out = 0, in = 0;
                    for (String other : members) {
                        if (other.equals(m)) continue;
                        out += weights.getOrDefault(m, Map.of()).getOrDefault(other, 0);
                        in  += weights.getOrDefault(other, Map.of()).getOrDefault(m, 0);
                    }
                    rank.put(m, (out - in) / (double) Math.max(1, out + in));
                }

                // Identify back-edges: from→to where rank(from) < rank(to) - threshold
                for (String from : new ArrayList<>(members)) {
                    for (String to : new ArrayList<>(graph.getOrDefault(from, Set.of()))) {
                        if (!members.contains(to)) continue;
                        if (rank.get(from) < rank.get(to) - RANK_THRESHOLD) {
                            String key = from + "\0" + to;
                            if (backEdgeKeys.add(key)) {
                                graph.get(from).remove(to);
                                changed = true;
                            }
                        }
                    }
                }
            }
        }

        // Assign package levels: SCC-collapsed DAG → longest-path
        List<StronglyConnectedComponent> sccs = new TarjanSCCFinder(graph).findSCCs();
        Map<String, StronglyConnectedComponent> pkgToScc = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) {
            for (String m : scc.getMembers()) pkgToScc.put(m, scc);
        }

        // SCC dependency graph (between SCC IDs)
        Map<Integer, Set<Integer>> sccDeps = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) {
            sccDeps.put(scc.getId(), new HashSet<>());
            for (String m : scc.getMembers()) {
                for (String to : graph.getOrDefault(m, Set.of())) {
                    StronglyConnectedComponent toScc = pkgToScc.get(to);
```

```java
                    if (toScc != null && toScc.getId() != scc.getId()) {
                        sccDeps.get(scc.getId()).add(toScc.getId());
                    }
                }
            }
        }

        // Longest-path levels on the SCC-collapsed DAG. The previous
        // "childPkgUsedLevel" lift (which inflated a parent's level by
        // the class levels its child packages used) is gone -- layout
        // positioning lives on localLayerIndex now, so architectureLevel
        // can be a direct dependency-chain depth.
        Map<Integer, Integer> sccLevels = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) sccLevels.put(scc.getId(), 0);
        boolean lvlChanged = true;
        while (lvlChanged) {
            lvlChanged = false;
            for (StronglyConnectedComponent scc : sccs) {
                int maxDep = -1;
                for (int depId : sccDeps.getOrDefault(scc.getId(), Set.of())) {
                    maxDep = Math.max(maxDep, sccLevels.getOrDefault(depId, 0));
                }
                int newLevel = maxDep >= 0 ? maxDep + 1 : 0;
                if (sccLevels.get(scc.getId()) != newLevel) {
                    sccLevels.put(scc.getId(), newLevel);
                    lvlChanged = true;
                }
            }
        }

        // Apply levels to all packages
        Map<String, DomainModel.CalculatedElementInfo> pkgInfos = model.getAllPackages();
        for (StronglyConnectedComponent scc : sccs) {
            int level = sccLevels.get(scc.getId());
            for (String member : scc.getMembers()) {
                DomainModel.CalculatedElementInfo pkg = pkgInfos.get(member);
                if (pkg != null) pkg.setArchitectureLevel(level);
            }
        }

        // Store the weighted graph and identified back-edges in the model
        model.setPackageEdgeWeights(weights);
        model.setPackageBackEdges(backEdgeKeys);

        // Populate package dependencies (unweighted) for reverse-dependency tracking
        for (Map.Entry<String, Map<String, Integer>> entry : weights.entrySet()) {
            DomainModel.CalculatedElementInfo pkg = pkgInfos.get(entry.getKey());
            if (pkg != null) entry.getValue().keySet().forEach(pkg::addDependency);
        }
    }

    /**
     * Builds the weighted inter-package dependency graph.
     * weight(P_A → P_B) = total method-call count from classes in P_A's subtree
     * to classes in P_B. Method calls are a stronger signal than distinct-class
     * counts: a class called hundreds of times clearly dominates one called once,
     * making SCC-breaking direction unambiguous. Intra-subtree calls are excluded.
     * Child-to-ancestor and parent-to-child edges are included.
     * Each call count is propagated to all ancestor packages up to the point where
     * the ancestor would enter the same subtree as the target.
     */
    private Map<String, Map<String, Integer>> buildWeightedPackageGraph(
```

```java
            DomainModel model, DependencyModel rawModel) {
        Map<String, Map<String, Integer>> weights = new HashMap<>();
        for (String pkg : model.getAllPackages().keySet()) weights.put(pkg, new LinkedHashMap<>());

        Set<String> allPkgNames = weights.keySet();

        for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
            String leafPkg = extractPackageName(cls.fullName);
            if (leafPkg == null || !allPkgNames.contains(leafPkg)) continue;

            // Count method calls per target package using raw model data
            Map<String, Integer> callCountPerPkg = new LinkedHashMap<>();
            DependencyModel.ClassInfo rawCls = rawModel.getClass(cls.fullName);
            if (rawCls != null) {
                for (DependencyModel.MethodInfo method : rawCls.methods.values()) {
                    for (Map.Entry<String, Integer> call : method.methodCalls.entrySet()) {
                        // Method call key format: "ownerClass.methodName"
                        // Find the target class by checking known dependencies
                        String calledKey = call.getKey();
                        for (String dep : cls.dependencies) {
                            if (calledKey.startsWith(dep + ".")) {
                                String toPkg = extractPackageName(dep);
                                if (toPkg != null && !leafPkg.equals(toPkg)
                                        && allPkgNames.contains(toPkg)) {
                                    callCountPerPkg.merge(toPkg, call.getValue(), Integer::sum);
                                }
                                break;
                            }
                        }
                    }
                }
            }
            // Fall back to 1 per target package for structural deps with no call data
            for (String dep : cls.dependencies) {
                String toPkg = extractPackageName(dep);
                if (toPkg != null && !leafPkg.equals(toPkg) && allPkgNames.contains(toPkg)) {
                    callCountPerPkg.putIfAbsent(toPkg, 1);
                }
            }

            // Propagate each weight up to ancestor packages. Parent->child
            // edges are no longer filtered here -- the rank-based SCC breaker
            // decides direction based on actual call-count weight. Layout
            // positioning is the LocalLayerCalculator’s job (step 6), so the
            // global package level can be a direct dependency-chain count.
            for (Map.Entry<String, Integer> entry : callCountPerPkg.entrySet()) {
                String toPkg = entry.getKey();
                int callCount = entry.getValue();
                String ancestor = leafPkg;
                while (ancestor != null && allPkgNames.contains(ancestor)) {
                    if (ancestor.equals(toPkg)) break;
                    weights.get(ancestor).merge(toPkg, callCount, Integer::sum);
                    ancestor = extractPackageName(ancestor);
                }
            }
        }
        return weights;
    }

    // ------------------------------------------------------------------------
    // Step 5 -- reverse dependencies
    // ------------------------------------------------------------------------
```

```java

    private void updateDependentRelationships(DomainModel model) {
        for (DomainModel.CalculatedElementInfo cls : model.getAllClasses().values()) {
            for (String dep : cls.dependencies) {
                DomainModel.CalculatedElementInfo d = model.getClass(dep);
                if (d != null) d.addDependent(cls.fullName);
            }
        }
        for (DomainModel.CalculatedElementInfo pkg : model.getAllPackages().values()) {
            for (String dep : pkg.dependencies) {
                DomainModel.CalculatedElementInfo d = model.getPackage(dep);
                if (d != null) d.addDependent(pkg.fullName);
            }
        }
    }

    // ------------------------------------------------------------------------
    // Utilities
    // ------------------------------------------------------------------------

    private static boolean isInSameSubtree(String a, String b) {
        return a.startsWith(b + ".") || b.startsWith(a + ".");
    }

    private static String extractPackageName(String className) {
        if (className == null || !className.contains(".")) return null;
        return className.substring(0, className.lastIndexOf('.'));
    }
}
```

The companion class `LocalLayerCalculator` implements Phase 3 (Step 6).

```java
package de.weigend.s202.domain.architecture;

import de.weigend.s202.domain.DomainModel;
import de.weigend.s202.domain.DomainModel.CalculatedElementInfo;
import de.weigend.s202.graph.StronglyConnectedComponent;
import de.weigend.s202.graph.TarjanSCCFinder;
import de.weigend.s202.reader.DependencyModel;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashSet;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 * Assigns a per-parent layer position ({@code localLayerIndex}) to every
 * element in a {@link DomainModel}. Each parent package is treated as a
 * self-contained scope: only class dependencies whose source <em>and</em>
 * target both live in the parent's subtree contribute to the sibling-only
 * weighted graph. Refs that leave the parent are ignored -- they belong to
 * the global {@code architectureLevel} via the package hierarchy.
 *
 * <p>Within each parent's sibling graph:
 * <ol>
 *   <li>Tarjan finds SCCs.</li>
 *   <li>SCCs of size &gt; 1 are broken by weight-based rank: a back-edge
 *       leaves the smaller-rank member, the dominant direction stays.</li>
 *   <li>Longest-path on the SCC-collapsed DAG assigns a {@code layer}
```

```java
 *       (= local layer index) per sibling. Layer 0 = bottom of the box,
 *       higher values = visually higher.</li>
 * </ol>
 *
 * <p>Single-child containers stay at layer 0 -- no graph, no work.
 *
 * <p>The algorithm intentionally mirrors the structure of
 * {@link LevelCalculator}'s package-level computation, but applied
 * locally and using a different graph. Local cycles are not surfaced
 * as tangles; the user-visible tangle list remains the global one.
 */
public class LocalLayerCalculator {

    private static final double RANK_THRESHOLD = 0.1;

    public void assign(DomainModel domain, DependencyModel rawModel) {
        Map<String, List<CalculatedElementInfo>> childrenByParent = groupChildrenByParent(domain);
        for (Map.Entry<String, List<CalculatedElementInfo>> entry : childrenByParent.entrySet()) {
            assignForParent(entry.getValue(), domain, rawModel);
        }
    }

    private void assignForParent(List<CalculatedElementInfo> siblings,
                                 DomainModel domain, DependencyModel rawModel) {
        if (siblings.size() <= 1) {
            return; // single child stays at layer 0
        }

        Set<String> siblingFqns = new LinkedHashSet<>();
        for (CalculatedElementInfo c : siblings) {
            siblingFqns.add(c.fullName);
        }

        Map<String, Map<String, Integer>> weights = buildSiblingGraph(siblingFqns, domain, rawModel);
        Map<String, Integer> layers = computeLayers(weights, siblingFqns);
        for (CalculatedElementInfo s : siblings) {
            s.setLocalLayerIndex(layers.getOrDefault(s.fullName, 0));
        }
    }

    /**
     * Build the weighted sibling-only edge map. For every class C in any
     * sibling's subtree, every dep into another sibling's subtree
     * contributes {@code callCount(C, dep)} weight; deps that leave the
     * parent's subtree or stay inside the same sibling are skipped.
     */
    private Map<String, Map<String, Integer>> buildSiblingGraph(Set<String> siblingFqns,
                                                                DomainModel domain,
                                                                DependencyModel rawModel) {
        Map<String, Map<String, Integer>> weights = new HashMap<>();
        for (String s : siblingFqns) {
            weights.put(s, new HashMap<>());
        }
        for (CalculatedElementInfo cls : domain.getAllClasses().values()) {
            String fromSibling = containingSibling(cls.fullName, siblingFqns);
            if (fromSibling == null) {
                continue;
            }
            for (String dep : cls.dependencies) {
                if (domain.getClass(dep) == null) {
                    continue; // external -- not in any sibling
                }
```

```java
                String toSibling = containingSibling(dep, siblingFqns);
                if (toSibling == null || toSibling.equals(fromSibling)) {
                    continue; // out of parent OR intra-sibling
                }
                int w = callCount(cls.fullName, dep, rawModel);
                weights.get(fromSibling).merge(toSibling, w, Integer::sum);
            }
        }
        return weights;
    }

    /**
     * Walk up the fqn package chain until the first ancestor that is one
     * of the siblings, or return {@code null} when none of the ancestors
     * is -- the class lives outside the parent's subtree.
     */
    private static String containingSibling(String fqn, Set<String> siblingFqns) {
        String current = fqn;
        while (current != null && !current.isEmpty()) {
            if (siblingFqns.contains(current)) {
                return current;
            }
            int dot = current.lastIndexOf('.');
            if (dot < 0) {
                return null;
            }
            current = current.substring(0, dot);
        }
        return null;
    }

    private static Map<String, List<CalculatedElementInfo>> groupChildrenByParent(DomainModel domain) {
        Map<String, List<CalculatedElementInfo>> result = new HashMap<>();
        for (CalculatedElementInfo cls : domain.getAllClasses().values()) {
            result.computeIfAbsent(parentOf(cls.fullName), k -> new ArrayList<>()).add(cls);
        }
        for (CalculatedElementInfo pkg : domain.getAllPackages().values()) {
            result.computeIfAbsent(parentOf(pkg.fullName), k -> new ArrayList<>()).add(pkg);
        }
        return result;
    }

    private static String parentOf(String fqn) {
        int dot = fqn.lastIndexOf('.');
        return dot < 0 ? "" : fqn.substring(0, dot);
    }

    /**
     * Method-call count from class {@code from} to class {@code to}.
     * Falls back to 1 for structural deps (extends/implements/field type)
     * where no method-call info exists -- same convention the global
     * weighted package graph uses.
     */
    private static int callCount(String from, String to, DependencyModel rawModel) {
        DependencyModel.ClassInfo cls = rawModel.getClass(from);
        if (cls == null) {
            return 1;
        }
        int count = 0;
        String prefix = to + ".";
        for (DependencyModel.MethodInfo method : cls.methods.values()) {
            for (Map.Entry<String, Integer> call : method.methodCalls.entrySet()) {
```

```java
            if (call.getKey().startsWith(prefix)) {
                count += call.getValue();
            }
        }
    }
    return count > 0 ? count : 1;
}

// ----------- SCC-collapsed longest path with rank-based break -----------

private Map<String, Integer> computeLayers(Map<String, Map<String, Integer>> weights,
                                           Set<String> nodes) {
    Map<String, Set<String>> graph = new HashMap<>();
    for (String n : nodes) {
        graph.put(n, new HashSet<>(weights.getOrDefault(n, Map.of()).keySet()));
    }

    // Identify and remove back-edges within SCCs based on weight rank.
    boolean changed = true;
    while (changed) {
        changed = false;
        for (StronglyConnectedComponent scc : new TarjanSCCFinder(graph).findSCCs()) {
            if (scc.getSize() < 2) {
                continue;
            }
            Set<String> members = scc.getMembers();
            Map<String, Double> rank = new HashMap<>();
            for (String m : members) {
                int out = 0;
                int in = 0;
                for (String other : members) {
                    if (other.equals(m)) {
                        continue;
                    }
                    out += weights.getOrDefault(m, Map.of()).getOrDefault(other, 0);
                    in  += weights.getOrDefault(other, Map.of()).getOrDefault(m, 0);
                }
                rank.put(m, (out - in) / (double) Math.max(1, out + in));
            }
            for (String from : new ArrayList<>(members)) {
                for (String to : new ArrayList<>(graph.getOrDefault(from, Set.of()))) {
                    if (!members.contains(to)) {
                        continue;
                    }
                    if (rank.get(from) < rank.get(to) - RANK_THRESHOLD) {
                        graph.get(from).remove(to);
                        changed = true;
                    }
                }
            }
        }
    }

    // SCC-collapsed longest-path level assignment.
    List<StronglyConnectedComponent> sccs = new TarjanSCCFinder(graph).findSCCs();
    Map<String, StronglyConnectedComponent> nodeToScc = new HashMap<>();
    for (StronglyConnectedComponent scc : sccs) {
        for (String m : scc.getMembers()) {
            nodeToScc.put(m, scc);
        }
    }
    Map<Integer, Set<Integer>> sccDeps = new HashMap<>();
```

```
        for (StronglyConnectedComponent scc : sccs) {
            sccDeps.put(scc.getId(), new HashSet<>());
            for (String m : scc.getMembers()) {
                for (String to : graph.getOrDefault(m, Set.of())) {
                    StronglyConnectedComponent toScc = nodeToScc.get(to);
                    if (toScc != null && toScc.getId() != scc.getId()) {
                        sccDeps.get(scc.getId()).add(toScc.getId());
                    }
                }
            }
        }
        Map<Integer, Integer> sccLayers = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) {
            sccLayers.put(scc.getId(), 0);
        }
        boolean lvlChanged = true;
        while (lvlChanged) {
            lvlChanged = false;
            for (StronglyConnectedComponent scc : sccs) {
                int maxDep = -1;
                for (int depId : sccDeps.get(scc.getId())) {
                    maxDep = Math.max(maxDep, sccLayers.get(depId));
                }
                int newLayer = maxDep >= 0 ? maxDep + 1 : 0;
                if (sccLayers.get(scc.getId()) != newLayer) {
                    sccLayers.put(scc.getId(), newLayer);
                    lvlChanged = true;
                }
            }
        }

        Map<String, Integer> result = new HashMap<>();
        for (StronglyConnectedComponent scc : sccs) {
            int layer = sccLayers.get(scc.getId());
            for (String m : scc.getMembers()) {
                result.put(m, layer);
            }
        }
        return result;
    }
}
```

Appendix C – Questions and Answers

Why does S202 need a separate hierarchy for packages?

Because classes and packages are ordered by different criteria. A class level is the depth of a node in the class dependency DAG after SCC handling: if class *A* depends on class *B*, then *A* belongs above *B* unless the edge is an explicit heuristic back-edge or both classes remain in the same SCC.

A package level answers a different question. Packages are containers, and a single package can contain many classes with dependencies in both directions. The package order therefore cannot be derived from the deepest class inside the package. Instead, S202 compares the weighted relations between packages: when the traffic from package P_A to package P_B dominates the reverse traffic, P_A is treated as the higher dependent package and P_B as the lower package it depends on. Equal or near-equal bidirectional traffic is kept as a cyclic peer relationship.

This is why class architecture levels and package architecture levels are computed on separate graphs. The separation prevents class depth from accidentally lifting containers and prevents package aggregation from changing the meaning of class-level dependency depth.

It also prevents a feedback loop. If package levels are derived from class levels and then feed back into class or layout ordering, each update can change the input of the next update. A package moves because one contained class moves; that package movement then changes sibling order or parent order; the changed order can expose another cycle or another dominant relation. Sorting and leveling algorithms then no longer operate on a stable graph. The result can depend on traversal order, classpath order, or on how often the loop is run. The system may converge accidentally, oscillate, or settle on a layout that is only an artifact of the update sequence. Separating the graphs makes each phase operate on fixed input and gives the invariants a well-defined result to check.

Why does S202 need both an architecture level and a local UI level?

Because these two numbers answer different questions. The architecture level of a class says how deep the class is in the global dependency graph: how many dependency steps separate it from the lower end of the class DAG after SCC handling. This is a system-wide analysis value.

The local layer index answers a layout question: where should this class or sub-package be placed relative to the other direct children of the same package? That question cannot be answered reliably from the global architecture level alone. A class can have a high global architecture level because it depends on deep chains elsewhere in the system, even though inside its own package it is not the highest local element. Conversely, a package-level coordinator may need to sit above a sub-package it calls into, even if their global architecture levels do not express that local relationship.

Using only the global DAG level for layout therefore over-constrains the picture. Elements can be pulled too high by dependencies outside their local container, and subsequent sorting passes try to repair a layout decision with the wrong input value. That is hard to make stable because the global level and the local sibling order are coupled through unrelated dependencies.

S202 keeps both values. The architecture level remains the semantic dependency depth used for global checks and reports. The local layer index is recomputed inside each package from the sibling-only graph and is used for visual ordering inside that package. The renderer can then place elements correctly within a container without changing what their global architecture level means.

Open Points Before Submission

- Keep the submission abstract to one page in the INSTICC template.